

```
In [1]: # 前述数据集采集, 模型构建过程
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets
from torchvision.transforms import ToTensor

training_data = datasets.FashionMNIST(
    root="E:\\pytorch\\dataset\\fashion_mnist\\train_data",
    train=True,
    download=False,
    transform=ToTensor()
)

test_data = datasets.FashionMNIST(
    root="E:\\pytorch\\dataset\\fashion_mnist\\test_data",
    train=False,
    download=False,
    transform=ToTensor()
)

train_dataloader = DataLoader(training_data, batch_size=64)
test_dataloader = DataLoader(test_data, batch_size=64)

class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits

model = NeuralNetwork()
```

```
In [2]: # 设置模型训练的超参数
```

```
learning_rate = 1e-3
batch_size = 64
epochs = 5
```

```
In [3]: def train_loop(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    # Set the model to training mode - important for batch normalization and dropout
    # Unnecessary in this situation but added for best practices
    model.train()
```

```

for batch, (X, y) in enumerate(dataloader):
    # Compute prediction and loss
    pred = model(X)
    loss = loss_fn(pred, y)

    # Backpropagation
    loss.backward()
    optimizer.step()
    optimizer.zero_grad()

    if batch % 100 == 0:
        loss, current = loss.item(), batch * batch_size + len(X)
        print(f"loss: {loss:>7f} [{current:>5d}/{size:>5d}]")

def test_loop(dataloader, model, loss_fn):
    # Set the model to evaluation mode - important for batch normalization and drop
    # Unnecessary in this situation but added for best practices
    model.eval()
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    test_loss, correct = 0, 0

    # Evaluating the model with torch.no_grad() ensures that no gradients are compu
    # also serves to reduce unnecessary gradient computations and memory usage for
    with torch.no_grad():
        for X, y in dataloader:
            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()

    test_loss /= num_batches
    correct /= size
    print(f"Test Error: \n Accuracy: {(100*correct)>0.1f}%, Avg loss: {test_loss:>

```

```

In [4]: loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

epochs = 10
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    train_loop(train_dataloader, model, loss_fn, optimizer)
    test_loop(test_dataloader, model, loss_fn)
print("Done!")

```

Epoch 1

loss: 2.315117 [64/60000]
loss: 2.293469 [6464/60000]
loss: 2.280816 [12864/60000]
loss: 2.270231 [19264/60000]
loss: 2.255151 [25664/60000]
loss: 2.227713 [32064/60000]
loss: 2.235583 [38464/60000]
loss: 2.205908 [44864/60000]
loss: 2.204734 [51264/60000]
loss: 2.169114 [57664/60000]

Test Error:

Accuracy: 44.9%, Avg loss: 2.164837

Epoch 2

loss: 2.174148 [64/60000]
loss: 2.158812 [6464/60000]
loss: 2.109697 [12864/60000]
loss: 2.125117 [19264/60000]
loss: 2.077729 [25664/60000]
loss: 2.017861 [32064/60000]
loss: 2.048726 [38464/60000]
loss: 1.975361 [44864/60000]
loss: 1.982532 [51264/60000]
loss: 1.909385 [57664/60000]

Test Error:

Accuracy: 53.0%, Avg loss: 1.907375

Epoch 3

loss: 1.935360 [64/60000]
loss: 1.898681 [6464/60000]
loss: 1.793692 [12864/60000]
loss: 1.838295 [19264/60000]
loss: 1.723887 [25664/60000]
loss: 1.674152 [32064/60000]
loss: 1.700725 [38464/60000]
loss: 1.603628 [44864/60000]
loss: 1.628692 [51264/60000]
loss: 1.522971 [57664/60000]

Test Error:

Accuracy: 61.8%, Avg loss: 1.542983

Epoch 4

loss: 1.601940 [64/60000]
loss: 1.562933 [6464/60000]
loss: 1.424599 [12864/60000]
loss: 1.498898 [19264/60000]
loss: 1.377374 [25664/60000]
loss: 1.364979 [32064/60000]
loss: 1.379701 [38464/60000]
loss: 1.308353 [44864/60000]
loss: 1.341567 [51264/60000]

loss: 1.236549 [57664/60000]
Test Error:
Accuracy: 64.1%, Avg loss: 1.270964

Epoch 5

loss: 1.341738 [64/60000]
loss: 1.321186 [6464/60000]
loss: 1.164804 [12864/60000]
loss: 1.270066 [19264/60000]
loss: 1.146000 [25664/60000]
loss: 1.158833 [32064/60000]
loss: 1.177737 [38464/60000]
loss: 1.121677 [44864/60000]
loss: 1.160462 [51264/60000]
loss: 1.067425 [57664/60000]
Test Error:
Accuracy: 65.1%, Avg loss: 1.098923

Epoch 6

loss: 1.164972 [64/60000]
loss: 1.165622 [6464/60000]
loss: 0.991091 [12864/60000]
loss: 1.125185 [19264/60000]
loss: 0.999024 [25664/60000]
loss: 1.017335 [32064/60000]
loss: 1.051307 [38464/60000]
loss: 1.001203 [44864/60000]
loss: 1.041592 [51264/60000]
loss: 0.961237 [57664/60000]
Test Error:
Accuracy: 66.0%, Avg loss: 0.986599

Epoch 7

loss: 1.040340 [64/60000]
loss: 1.063607 [6464/60000]
loss: 0.870745 [12864/60000]
loss: 1.028168 [19264/60000]
loss: 0.905422 [25664/60000]
loss: 0.917100 [32064/60000]
loss: 0.968278 [38464/60000]
loss: 0.922347 [44864/60000]
loss: 0.959618 [51264/60000]
loss: 0.890171 [57664/60000]
Test Error:
Accuracy: 67.2%, Avg loss: 0.910057

Epoch 8

loss: 0.948388 [64/60000]
loss: 0.992883 [6464/60000]
loss: 0.784269 [12864/60000]
loss: 0.959548 [19264/60000]
loss: 0.843455 [25664/60000]

```
loss: 0.844298 [32064/60000]
loss: 0.910209 [38464/60000]
loss: 0.869608 [44864/60000]
loss: 0.901437 [51264/60000]
loss: 0.839346 [57664/60000]
Test Error:
  Accuracy: 68.3%, Avg loss: 0.855494
```

Epoch 9

```
-----
loss: 0.877953 [ 64/60000]
loss: 0.940006 [ 6464/60000]
loss: 0.719859 [12864/60000]
loss: 0.908888 [19264/60000]
loss: 0.800324 [25664/60000]
loss: 0.790500 [32064/60000]
loss: 0.866624 [38464/60000]
loss: 0.833166 [44864/60000]
loss: 0.858750 [51264/60000]
loss: 0.800845 [57664/60000]
Test Error:
  Accuracy: 69.7%, Avg loss: 0.814598
```

Epoch 10

```
-----
loss: 0.822134 [ 64/60000]
loss: 0.897629 [ 6464/60000]
loss: 0.669938 [12864/60000]
loss: 0.870155 [19264/60000]
loss: 0.768212 [25664/60000]
loss: 0.749584 [32064/60000]
loss: 0.831802 [38464/60000]
loss: 0.806514 [44864/60000]
loss: 0.826177 [51264/60000]
loss: 0.770271 [57664/60000]
Test Error:
  Accuracy: 71.2%, Avg loss: 0.782414
```

Done!

In []: